Abdulhannan Ahmad

m.abdulhannan.ahmad@gmail.com | +44 7971257218
abdul.fyi | linkedin.com/in/abdulhannan-ahmad | github.com/Abdul-213

Summary

I'm a backend software engineer who enjoys solving complex problems, and creating tools that make technology more seamless and useful in everyday life. I'm particularly interested in AI-powered products, and the challenge that comes with building reliable, high-performance systems that scale. I thrive in collaborative, fast-moving environments where I can learn quickly and ship impactful solutions.

Skills

Languages	Python | Java | Go | Bash | SQL |
Infrastructure	AWS | Azure | Cloudformation | Terraform | Kubernetes
Tools/Frameworks	Cloudflare | Docker | Elasticsearch | Flask | FastAPI | MongoDB | PagerDuty
Practices	Agile (SAFe) | Chaos Engineering | CI/CD | DevOps | Git | Performance Testing
Observability	Datadog | Grafana | Loki | Prometheus | Tempo

Experience

Vodafone — **London, UK**

Platform Engineer | Cloud Engineering — *July 2025 - Present*

- Designed and deployed secure cross-region connectivity between internal platforms using private API Gateway endpoints and AWS PrivateLink, establishing a reusable pattern.
- Led the migration to Cloudflare, automating resource provisioning with Terraform and creating modules, enabling teams to self-serve configuration changes.
- Delivered a platform enabling external partners to securely access internal APIs, automating provisioning of an API Gateway with Lambda Authoriser using Terraform and securing traffic through Cloudflare Access, establishing a repeatable pattern for third-party onboarding.
- Built a conversational AI solution to handle large volumes of customer queries using Azure AI Foundry, partnering with teams to deliver production-ready AI services.

Software Engineer | Performance and Chaos Engineering — *Aug 2022 - July 2025*

- Led the development of a performance testing framework automating performance tests for 100+ microservices, incorporating custom tooling with Locust (Python), Wiremock (Java) and ChaosToolkit (Python), reducing manual overhead from days to minutes.
- Led the initative to scale performance testing to simulate 4000+ system wide RPS, partnering across teams to identify bottlenecks in database connection pools, lambda concurrency and autoscaling configurations
- Halved performance test execution time by redesigning test orchestration, allowing developers to validate their results in 30 minutes instead of an hour.
- Built and maintained multiple CI/CD pipelines using Terraform, CloudFormation, YAML, and Bash to provision AWS infrastructure, supporting automated deployments across development and production environments.
- Standardised mocking infrastructure by consolidating 20 disparate mocking solutions to a single platform, eliminating the maintenance burden and ensuring mocking consistency across multiple teams.
- Led incident response as an on-call engineer and incident commander, diagnosing and resolving production issues through effective triage with DataDog, and coordinating follow-up postmortems to identify the root cause.
- Designed and led chaos engineering war games, simulating realistic production failures for SREs and developers, validating resiliency and measuring incident response effectiveness, driving operational improvements.
- Mentored new graduates and team members through structured onboarding, pairing sessions and technical guidance, accelerating their ramp-up time

Education

BSc Computer Science with Digital Technology Partnership, First Class — **Sep 2018 - Aug 2022**

University of Birmingham — *Birmingham, UK*

Projects

Riftcodex

- Developed a free REST API for Riftbound, a trading card game, using Cloudflare Workers to host the website and Python (Fast API) for the backend. Added observability using Prometheus, Loki, and Tempo, with Grafana dashboards for metrics, logs, and traces. As of April 2026, there are 12,000 monthly visitors and 296,000 monthly requests to the API.

Search Engine

- Developed a search engine for Magic: The Gathering using Retrieval-Augmented Generation (RAG), enabling users to search for cards using natural language queries. Built the backend in Go, hosted the application on Cloudflare Workers and used Cloudflare Vectorize to store the embeddings for over 30,000 cards.